🔥 PySpark 80/20 Guide for College Students

with SQL side by side

> **Learn the 20% of PySpark that you will use 80% of the time.** Eat it (read), sweat it (practice), enjoy it (build things). 🍽️💪😄

> **What is new in this edition:** every PySpark example now has its **SQL twin** on the same `employees` and `depts` tables. If you already know SQL, read the SQL block first, then see how the same idea is written in PySpark. Every output shown here was produced by running the code on real Spark.

📌 How to use this guide

1. Read one section (10 minutes).
2. **Type the code yourself**. Do not copy-paste.
3. For every example, read the **SQL block** first (you know it), then the **PySpark block** (new). Notice the one-to-one match.
4. Do the "Try it" at the end of each section. The answers are given in **both** languages.
5. Finish with the **Interview Questions** and **Practice Set**.

Tip for teachers: each section = one class of ~30-40 minutes.

Colour code used everywhere: the blue tag is **PySpark**, the green tag is **SQL**, the grey box is the **Output**.

🗺️ The 80/20 Map (what actually matters)

#	Topic	Why it matters	SQL cousin	Time
1	Spark mental model (lazy, transformations vs actions)	Explains *everything* else	`EXPLAIN`, query plan	20 min
2	Read / write data	Every job starts and ends here	`CREATE TABLE`, `INSERT`	15 min
3	`select`, `filter`, `withColumn`, `cast`	60% of daily code	`SELECT`, `WHERE`, `CASE`, `CAST`	30 min
4	Nulls and duplicates	Real data is dirty	`IS NULL`, `COALESCE`, `DISTINCT`	15 min
5	`groupBy` + `agg`	Reports and summaries	`GROUP BY`, `HAVING`	20 min
6	Joins	Combining tables	`JOIN ... ON`	30 min

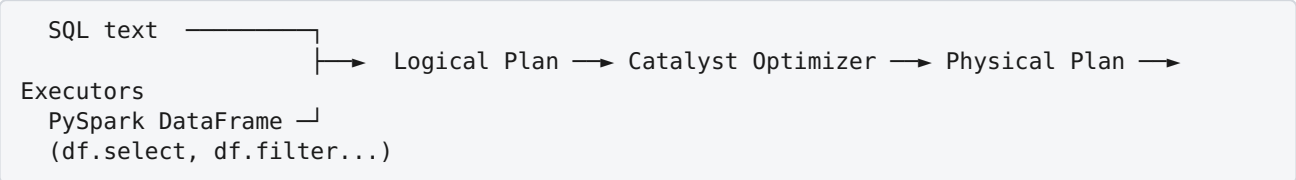
#	Topic	Why it matters	SQL cousin	Time
7	Window functions	Ranking, running totals, "top N per group"	`OVER (PARTITION BY ...)`	30 min
8	Built-in functions vs UDF	Speed and correctness	SQL functions, `CASE`	15 min
9	Performance basics (partitions, cache, broadcast)	Interview favourite	Hints, `CACHE TABLE`	30 min

Skip for now: RDD low-level APIs, MLlib, GraphX, custom partitioners, Structured Streaming internals. Learn them later *when needed*.

🔗 PySpark and SQL: One Engine, Two Languages

🧠 The big idea

In Spark, **SQL and the DataFrame API are two front doors to the same engine**. Whatever you write is converted into the same *logical plan*, optimized by the same **Catalyst optimizer**, and run by the same executors. So every SQL query you already know has a PySpark twin, and the performance is the same.



> 💡 **Bridge between the two worlds:** `df.createOrReplaceTempView("emp")` lets you query a DataFrame with SQL. `spark.sql("SELECT ...")` returns a DataFrame again. You can mix both in one program.

🍽️ Our two practice tables (same data, both languages)

`emp` (the employees table) has 8 rows. The row `(2, Ravi)` appears **twice on purpose**, and Tom has a **NULL** salary. Asha, Meena and Sara have **NULL** manager.

emp_id	name	dept	salary	join_date	manager_id
1	Asha	IT	60000	2021-03-01	NULL
2	Ravi	IT	75000	2020-06-15	1
3	Meena	HR	50000	2019-01-10	NULL
4	John	HR	52000	2022-09-20	3
5	Sara	Finance	80000	2018-11-05	NULL

emp_id	name	dept	salary	join_date	manager_id
6	Kiran	Finance	80000	2021-07-30	5
7	Tom	IT	NULL	2023-01-12	1
2	Ravi	IT	75000	2020-06-15	1 ← duplicate row!

`depts` (the department table) has 4 rows. Note that **Legal has no employees**.

dept	city
IT	Bangalore
HR	Hyderabad
Finance	Mumbai
Legal	Delhi

✨ SQL cheat-map: what you write in SQL → what you write in PySpark

SQL clause / function	PySpark method / function
`SELECT a, b`	`df.select("a", "b")`
`SELECT x AS y`	`F.col("x").alias("y")` or `df.withColumnRenamed("x","y")`
`WHERE cond`	`df.filter(cond)` or `df.where(cond)`
`AND`, `OR`, `NOT`	`&`, `|`, `~` (with parentheses!)
`IN (...)`, `LIKE`, `BETWEEN`	`.isin(...)`, `.like(...)`, `.between(a, b)`
`IS NULL`, `IS NOT NULL`	`.isNull()`, `.isNotNull()`
`CASE WHEN ... THEN ... ELSE ... END`	`F.when(...).when(...).otherwise(...)`
`CAST(x AS DOUBLE)`	`F.col("x").cast("double")`
`COALESCE(a, b)`	`F.coalesce("a", "b")`
`DISTINCT`	`df.distinct()` or `df.dropDuplicates([...])`
`GROUP BY k` + aggregates	`df.groupBy("k").agg(...)`
`HAVING cond`	`.filter(cond)` placed *after* `.agg(...)`
`ORDER BY x DESC`	`df.orderBy(F.col("x").desc())`
`LIMIT n`	`df.limit(n)` (or `df.show(n)` to just display)
`JOIN ... ON a.k = b.k`	`df.join(other, on="k", how="inner")`
`UNION ALL` / `UNION`	`df1.union(df2)` / `df1.union(df2).distinct()`
`f() OVER (PARTITION BY .. ORDER BY ..)`	`F.f().over(Window.partitionBy(..).orderBy(..))`

SQL clause / function	PySpark method / function
`WITH cte AS (...)`	a variable: `cte = df.filter(...)`
`CREATE VIEW v AS ...`	`df.createOrReplaceTempView("v")`

⏱️ Order of execution: the one thing that confuses SQL people

In SQL you *write* `SELECT` first, but the engine *runs* it almost last. In PySpark you write the steps in the **order they run**, so many people find it easier to read.

```
SQL you write :  SELECT .. FROM .. WHERE .. GROUP BY .. HAVING .. ORDER BY .. LIMIT
SQL engine runs: FROM -> WHERE -> GROUP BY -> HAVING -> window -> SELECT -> DISTINCT ->
ORDER BY -> LIMIT

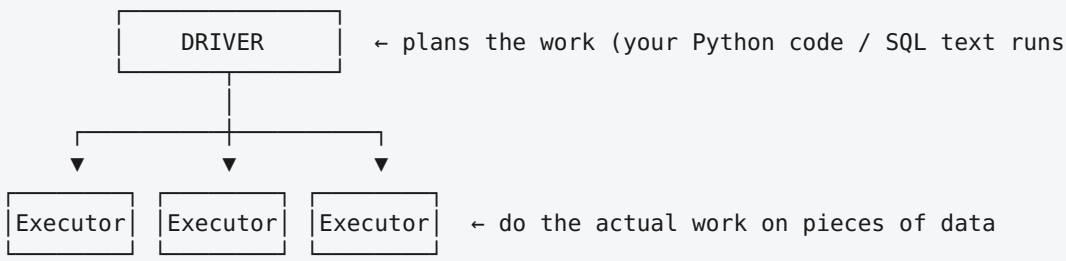
PySpark you write (= runs):
df.filter(..).groupBy(..).agg(..).filter(..).select(..).distinct().orderBy(..).limit(..)
```

> ⭐ **This is why** you cannot use a `SELECT` alias inside `WHERE` in classic SQL (the alias does not exist yet when `WHERE` runs). Newer Spark versions relax this for aliases in the same `SELECT`, but do not rely on it in other databases.

1️⃣ The Mental Model: Why Spark Exists

🍕 The pizza-shop analogy

- One chef (Pandas, or a single-server SQL database) makes 10 pizzas fine. But 10 million pizzas? One chef dies.
- Spark = **a kitchen with many chefs** (executors) and **one manager** (driver) who splits the work.



🔑 The 3 ideas that explain 80% of Spark

Idea 1: Data is split into partitions

Your big table is cut into pieces. Each piece is processed in parallel. *(SQL analogy: think of a table split across many disks, each scanned by a different worker.)*

Idea 2: Transformations are lazy, actions are eager

Type	Meaning	PySpark examples	SQL view
Transformation (lazy)	"Here is my *plan*." Nothing runs yet.	`select`, `filter`, `withColumn`, `join`, `groupBy`	The query text you write (`SELECT ... WHERE ...`) when used through `spark.sql(...)`: it builds a DataFrame but does not run yet
Action (eager)	"Now actually *do it* and give me a result."	`show`, `count`, `collect`, `write`, `take`	Pressing **Run** in a SQL notebook, or `.show()` on the result of `spark.sql(...)`, or `INSERT` / `CREATE TABLE AS`

Like ordering at a restaurant: writing the order = transformation. The waiter shouting it to the kitchen = action. 🧾➡️👨‍🍳

Idea 3: Shuffle is expensive

Moving data between machines (for `groupBy`, `join`, `orderBy`, `distinct`) is the slowest part of Spark. Good Spark = **fewer shuffles**. In SQL these are exactly `GROUP BY`, `JOIN`, `ORDER BY`, `DISTINCT`.

🧪 See laziness yourself

PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder.appName("learn").master("local[*]").getOrCreate()

df  = spark.range(1_000_000)                          # nothing runs
df2 = df.filter(F.col("id") % 2 == 0)                 # still nothing runs
df3 = df2.withColumn("sq", F.col("id")**2)            # still nothing!

df3.count()    # <- ACTION. Now Spark runs everything.
```

SQL

```
-- Same idea in SQL (run through spark.sql(...) or a SQL notebook cell)
SELECT id, POWER(id, 2) AS sq     -- the table "range(1000000)" is built in
FROM   range(1000000)
WHERE  id % 2 = 0;
-- spark.sql("...") alone is lazy. It only runs when you call an action:

SELECT COUNT(*) AS c
FROM  (SELECT id, POWER(id, 2) AS sq FROM range(1000000) WHERE id % 2 = 0);
```

Output

```
+------+
|c     |
+------+
|500000|
+------+
```

> 💡 Spark also **optimizes the whole plan** before running (Catalyst optimizer). That is the benefit of laziness. And because SQL and DataFrames share the optimizer, **both give the same plan**.

PySpark

```
df3.explain()
```

SQL

```
EXPLAIN
SELECT id, POWER(id, 2) AS sq
FROM   range(1000000)
WHERE  id % 2 = 0;
```

Output

```
== Physical Plan ==
*(1) Project [id#910L, POWER(cast(id#910L as double), 2.0) AS sq#905]
+- *(1) Filter ((id#910L % 2) = 0)
   +- *(1) Range (0, 1000000, step=1, splits=2)
```

Read the plan **from the bottom up**: Range (create ids) → Filter (keep even) → Project (compute `sq`). Notice that `id ** 2` in PySpark and `POWER(id, 2)` in SQL both become `POWER(cast(id as double), 2.0)`: the result is a **double**, in both languages.

Try it: Add `df3.explain()` before `count()` and read the plan. Then run the SQL `EXPLAIN` and compare.

2️⃣ Setup, DataFrames, Reading & Writing

Install and start

```
pip install pyspark
```

PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *

spark = (SparkSession.builder
         .appName("college-demo")
         .master("local[*]")      # use all CPU cores of your laptop
         .getOrCreate())
```

> ⭐ **Always** write `from pyspark.sql import functions as F` and use `F.col`, `F.sum`, etc. It avoids clashing with Python's `sum`, `max`, `min`. *(SQL has no such problem: `SUM` is just `SUM`.)*

Our practice dataset (use this through the whole guide)

PySpark

```
employees = [
    (1, "Asha",  "IT",      60000, "2021-03-01", None),
    (2, "Ravi",  "IT",      75000, "2020-06-15", 1),
    (3, "Meena", "HR",      50000, "2019-01-10", None),
    (4, "John",  "HR",      52000, "2022-09-20", 3),
    (5, "Sara",  "Finance", 80000, "2018-11-05", None),
    (6, "Kiran", "Finance", 80000, "2021-07-30", 5),
    (7, "Tom",   "IT",      None,  "2023-01-12", 1),
    (2, "Ravi",  "IT",      75000, "2020-06-15", 1),   # duplicate row!
]
cols = ["emp_id", "name", "dept", "salary", "join_date", "manager_id"]
df = spark.createDataFrame(employees, cols)

df.show()
df.printSchema()

df.createOrReplaceTempView("emp")      # <- now SQL can see it
```

SQL

```
-- Option A: Spark SQL (creates a temp view, no files needed)
CREATE OR REPLACE TEMP VIEW emp AS
SELECT * FROM VALUES
  (1, 'Asha',  'IT',      60000, '2021-03-01', NULL),
  (2, 'Ravi',  'IT',      75000, '2020-06-15', 1),
  (3, 'Meena', 'HR',      50000, '2019-01-10', NULL),
  (4, 'John',  'HR',      52000, '2022-09-20', 3),
  (5, 'Sara',  'Finance', 80000, '2018-11-05', NULL),
  (6, 'Kiran', 'Finance', 80000, '2021-07-30', 5),
  (7, 'Tom',   'IT',      NULL,  '2023-01-12', 1),
  (2, 'Ravi',  'IT',      75000, '2020-06-15', 1)      -- duplicate row!
AS t(emp_id, name, dept, salary, join_date, manager_id);

SELECT * FROM emp;          -- like df.show()
DESCRIBE emp;               -- like df.printSchema()
```

SQL (classic)

```
-- Option B: classic SQL (MySQL / PostgreSQL / SQL Server style)
CREATE TABLE emp (
  emp_id INT, name VARCHAR(50), dept VARCHAR(50),
  salary INT, join_date DATE, manager_id INT
);
INSERT INTO emp VALUES
  (1,'Asha','IT',60000,'2021-03-01',NULL),
  (2,'Ravi','IT',75000,'2020-06-15',1),
  (3,'Meena','HR',50000,'2019-01-10',NULL),
  (4,'John','HR',52000,'2022-09-20',3),
  (5,'Sara','Finance',80000,'2018-11-05',NULL),
  (6,'Kiran','Finance',80000,'2021-07-30',5),
  (7,'Tom','IT',NULL,'2023-01-12',1),
  (2,'Ravi','IT',75000,'2020-06-15',1);
```

Output

```
emp_id  name   dept     salary  join_date   manager_id
DESCRIBE emp ->  emp_id int | name string | dept string | salary int
                 join_date string | manager_id int
```

PySpark

```
depts = spark.createDataFrame(
    [("IT", "Bangalore"), ("HR", "Hyderabad"), ("Finance", "Mumbai"), ("Legal", "Delhi")],
    ["dept", "city"]
)
depts.createOrReplaceTempView("depts")
```

SQL

```
CREATE OR REPLACE TEMP VIEW depts AS
SELECT * FROM VALUES
  ('IT','Bangalore'), ('HR','Hyderabad'), ('Finance','Mumbai'), ('Legal','Delhi')
AS t(dept, city);
```

Reading data (the 4 you will use 95% of the time)

PySpark

```
# CSV
df = spark.read.csv("data.csv", header=True, inferSchema=True)

# JSON
df = spark.read.json("data.json")

# Parquet (the best format for big data)
df = spark.read.parquet("data.parquet")

# Table
df = spark.table("my_table")
```

SQL

```
-- CSV (give options through a temp view, so header and schema are handled)
CREATE OR REPLACE TEMP VIEW csv_df
USING csv OPTIONS (path 'data.csv', header 'true', inferSchema 'true');
SELECT * FROM csv_df;

-- JSON
SELECT * FROM json.`data.json`;

-- Parquet (the best format for big data)
SELECT * FROM parquet.`data.parquet`;

-- Table
SELECT * FROM my_table;
```

> ⚠️ **Watch out:** `SELECT * FROM csv.`data.csv`` (direct path) cannot take options. It reads the header line as a normal row and names the columns `_c0`, `_c1` ... That is why the CREATE TEMP VIEW ... OPTIONS (header 'true')` form is better for CSV.

⚠️ `inferSchema=True`: good for learning, bad for production

It reads the data twice and can guess wrong. In real projects, give a schema:

PySpark

```
schema = StructType([
    StructField("emp_id", IntegerType(), True),
    StructField("name",   StringType(),  True),
    StructField("dept",   StringType(),  True),
    StructField("salary", IntegerType(), True),
])
df = spark.read.csv("data.csv", header=True, schema=schema)
```

SQL

```
-- SQL: list the column names and types yourself
CREATE OR REPLACE TEMP VIEW csv_df (
  emp_id INT, name STRING, dept STRING, salary INT
)
USING csv OPTIONS (path 'data.csv', header 'true');
```

`StructField(name, type, nullable)` in PySpark is the same thing as `column_name TYPE` in a SQL `CREATE TABLE` (and `True` = the column allows NULL).

Writing data

PySpark

```
df.write.mode("overwrite").parquet("out/employees")

# partition by a column (creates folders dept=IT/, dept=HR/ ...)
df.write.mode("overwrite").partitionBy("dept").parquet("out/by_dept")
```

SQL

```
-- write straight to a folder as Parquet
INSERT OVERWRITE DIRECTORY 'out/employees'
USING parquet
SELECT * FROM emp;

-- partitioned table (creates folders dept=IT/, dept=HR/ ...)
CREATE TABLE by_dept
USING parquet
PARTITIONED BY (dept)
AS SELECT * FROM emp;
```

> 📝 In a partitioned table Spark moves the **partition column to the end** of the table. If you later `INSERT INTO` it, list the columns explicitly with `dept` **last**: `INSERT INTO by_dept SELECT emp_id, name, salary, join_date, manager_id, dept FROM emp.`

Mode	What it does	SQL equivalent
`overwrite`	Replace existing data	`INSERT OVERWRITE TABLE t SELECT ...` (for Delta tables also `CREATE OR REPLACE TABLE t AS SELECT ...`)
`append`	Add to existing data	`INSERT INTO t SELECT ...`
`ignore`	Do nothing if data exists	`CREATE TABLE IF NOT EXISTS t AS SELECT ...`

Mode	What it does	SQL equivalent
`error` (default)	Fail if data exists	`CREATE TABLE t AS SELECT ...` (fails if `t` already exists)

I tested these: a plain `CREATE TABLE ... AS SELECT` on an existing table fails with *"already exists"*, `IF NOT EXISTS` silently does nothing, and `INSERT OVERWRITE` replaces the rows. Also note that `CREATE OR REPLACE TABLE ... AS SELECT` works on **Delta** tables (Databricks) but **not** on plain Parquet tables.

Why Parquet?

- **Columnar**: reads only the columns you need.
- **Compressed**: small files.
- **Stores schema**: no guessing.

> CSV = a handwritten notebook 📓. Parquet = a well-organized library 📚.

3️⃣ The Daily Toolkit: `select`, `filter`, `withColumn`

select: pick columns (SQL: `SELECT`)

PySpark

```
df.select("name", "salary").show()

df.select(F.col("name"),
          (F.col("salary") * 1.1).alias("salary_after_hike")).show()
```

SQL

```
SELECT name, salary
FROM   emp;

SELECT name,
       salary * 1.1 AS salary_after_hike
FROM   emp;
```

Output

```
+-----+-----------------+
|name |salary_after_hike|
+-----+-----------------+
|Asha |66000.0          |
|Ravi |82500.0          |
|Meena|55000.0          |
+-----+-----------------+   (first 3 rows shown)
```

`.alias("x")` in PySpark = `AS x` in SQL. And `F.col("salary")` is simply "the column called salary", exactly like writing `salary` in SQL.

filter / where: pick rows (they are identical) (SQL: `WHERE`)

PySpark

```
df.filter(F.col("salary") > 60000).show()

# multiple conditions: use  &  |  ~  and ALWAYS use parentheses
df.filter((F.col("dept") == "IT") & (F.col("salary") > 60000)).show()

# in-list and pattern
df.filter(F.col("dept").isin("IT", "HR")).show()
df.filter(F.col("name").like("A%")).show()
```

SQL

```
SELECT * FROM emp WHERE salary > 60000;

-- multiple conditions: AND / OR / NOT  (note: single "=" for equality)
SELECT * FROM emp WHERE dept = 'IT' AND salary > 60000;

-- in-list and pattern
SELECT * FROM emp WHERE dept IN ('IT', 'HR');
SELECT * FROM emp WHERE name LIKE 'A%';
```

Idea	SQL	PySpark
equal	`dept = 'IT'` (one `=`)	`F.col("dept") == "IT"` (two `==`)
not equal	`dept <> 'IT'` or `!=`	`F.col("dept") != "IT"`
and / or / not	`AND`, `OR`, `NOT`	`&`, `|`, `~`
between	`salary BETWEEN 50000 AND 60000`	`F.col("salary").between(50000, 60000)`
in list	`dept IN ('IT','HR')`	`F.col("dept").isin("IT","HR")`

> ❌ **Common mistake:** using `and` / `or` instead of `&` / `|`. Python's `and` does not work on columns. In SQL you *do* write `AND` / `OR`, so this one is a trap only in PySpark. Also remember the parentheses: `&` binds tighter than `>` in Python.

withColumn: add or change a column (SQL: an expression in `SELECT`)

PySpark

```
df = (df
      .withColumn("bonus", F.col("salary") * 0.10)
      .withColumn("join_date", F.to_date("join_date", "yyyy-MM-dd"))
      .withColumn("join_year", F.year("join_date")))
```

SQL

```
WITH step1 AS (
  SELECT emp_id, name, dept, salary,
         to_date(join_date, 'yyyy-MM-dd') AS join_date,   -- changed column
         manager_id,
         salary * 0.10                    AS bonus        -- new column
  FROM emp
)
SELECT *, year(join_date) AS join_year                    -- new column using the one above
FROM   step1;
```

Output

```
+------+-----+----+------+----------+----------+-------+---------+
|emp_id|name |dept|salary|join_date |manager_id|bonus  |join_year|
+------+-----+----+------+----------+----------+-------+---------+
|1     |Asha |IT  |60000 |2021-03-01|NULL      |6000.00|2021     |
|2     |Ravi |IT  |75000 |2020-06-15|1         |7500.00|2020     |
|3     |Meena|HR  |50000 |2019-01-10|NULL      |5000.00|2019     |
+------+-----+----+------+----------+----------+-------+---------+  (3 rows shown)
```

Why the CTE? Each `withColumn` can use the column created by the previous one (`join_year` uses the new `join_date`). In classic SQL you cannot reuse an alias in the same `SELECT`, so you stack steps with `WITH ... AS (...)` or a subquery. Each `.withColumn(...)` ≈ one more step. Also note: SQL has no "change one column in place"; you re-list the columns (Databricks and newer Spark also allow `SELECT * EXCEPT (col)` to skip listing).

when / otherwise: Spark's if-else (SQL: `CASE WHEN`)

PySpark

```
df = df.withColumn(
    "level",
    F.when(F.col("salary") >= 75000, "Senior")
     .when(F.col("salary") >= 55000, "Mid")
     .otherwise("Junior")
)
df.select("name", "salary", "level").show()
```

SQL

```
SELECT name, salary,
       CASE WHEN salary >= 75000 THEN 'Senior'
            WHEN salary >= 55000 THEN 'Mid'
            ELSE                      'Junior'
       END AS level
FROM emp;
```

Output

```
+-----+------+------+
|name |salary|level |
+-----+------+------+
|Asha |60000 |Mid   |
|Ravi |75000 |Senior|
|Kiran|80000 |Senior|
|Tom  |NULL  |Junior|     <- NULL fails both tests, so it falls to otherwise / ELSE
+-----+------+------+       (de-duplicated rows, some shown)
```

`F.when(cond, value)` = `WHEN cond THEN value`. `.otherwise(v)` = `ELSE v`. The first matching branch wins, in both languages. **Tom's NULL salary** becomes *Junior*: a comparison with NULL is never true, so Spark moves on to `otherwise`. Handle NULL first if you do not want that.

cast, rename, drop

PySpark

```
df = df.withColumn("salary", F.col("salary").cast("double"))
df = df.withColumnRenamed("dept", "department")
df = df.drop("manager_id")
```

SQL

```
SELECT emp_id, name,
       dept AS department,                 -- rename = AS
       CAST(salary AS DOUBLE) AS salary,   -- cast
       join_date                           -- "drop manager_id" = just do not list it
FROM emp;
```

Sorting (SQL: `ORDER BY`)

PySpark

```
df.orderBy(F.col("salary").desc()).show()
df.orderBy("dept", F.col("salary").desc()).show()
```

SQL

```
SELECT * FROM emp ORDER BY salary DESC;
SELECT * FROM emp ORDER BY dept, salary DESC;
```

Output

```
ORDER BY dept, salary DESC   (de-duplicated, selected columns)
+-----+-------+------+
|name |dept   |salary|
+-----+-------+------+
|Kiran|Finance|80000 |
|Sara |Finance|80000 |
|John |HR     |52000 |
|Meena|HR     |50000 |
|Ravi |IT     |75000 |
|Asha |IT     |60000 |
|Tom  |IT     |NULL  |
+-----+-------+------+
```

> 📝 **Where do NULLs go?** In Spark, `ASC` puts NULLs **first** and `DESC` puts NULLs **last** (the same default in both SQL and PySpark). To control it: `ORDER BY salary DESC NULLS LAST` in SQL, `F.col("salary").desc_nulls_last()` in PySpark. Other databases have different defaults, so be explicit when you move between them.

Handy string and date functions

PySpark

```
F.upper("name"), F.lower("name"), F.trim("name")
F.concat_ws(" - ", "name", "dept")
F.substring("name", 1, 3)
F.regexp_replace("name", "[aeiou]", "*")

F.current_date()
F.datediff(F.current_date(), "join_date")
F.date_add("join_date", 30)
F.months_between(F.current_date(), "join_date")
```

SQL

```
upper(name), lower(name), trim(name)
concat_ws(' - ', name, dept)
substring(name, 1, 3)
regexp_replace(name, '[aeiou]', '*')

current_date()
datediff(current_date(), join_date)
date_add(join_date, 30)
months_between(current_date(), join_date)
```

Output

```
name = 'Ravi', dept = 'IT', join_date = '2020-06-15'
upper -> RAVI | lower -> ravi | concat_ws -> Ravi - IT
substring(name,1,3) -> Rav | regexp_replace -> R*v*
date_add(join_date, 30) -> 2020-07-15
```

In Spark the **function names are the same** in SQL and in `F.`; the only difference is you write them inside a string (SQL) or call them as Python functions (`F.`). `datediff(end, start)` takes the **later date first** in both.

Try it: Create a column `tenure_years` = years since `join_date`, rounded to 1 decimal.

PySpark

```
df = df.withColumn(
    "tenure_years",
    F.round(F.months_between(F.current_date(), F.to_date("join_date")) / 12, 1)
)
```

SQL

```
SELECT name, join_date,
       ROUND(months_between(current_date(), to_date(join_date)) / 12, 1) AS tenure_years
FROM   emp;
```

Output

```
+----+----------+------------+
|name|join_date |tenure_years|
+----+----------+------------+
|Asha|2021-03-01|5.7         |
|Ravi|2020-06-15|6.4         |
+----+----------+------------+   (values depend on today's date)
```

4️⃣ Dirty Data: Nulls and Duplicates

Real data is never clean. This is a **top interview topic**.

Nulls

PySpark

```
# find nulls
df.filter(F.col("salary").isNull()).show()
```

SQL

```
-- find nulls
SELECT * FROM emp WHERE salary IS NULL;
```

Output

```
+------+----+----+------+----------+----------+
|emp_id|name|dept|salary|join_date |manager_id|
+------+----+----+------+----------+----------+
|7     |Tom |IT  |NULL  |2023-01-12|1         |
+------+----+----+------+----------+----------+
```

PySpark

```
# count nulls per column
df.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in df.columns]).show()
```

SQL

```
-- count nulls per column (SQL has no loop, so you list the columns)
SELECT SUM(CASE WHEN emp_id     IS NULL THEN 1 ELSE 0 END) AS emp_id,
       SUM(CASE WHEN name       IS NULL THEN 1 ELSE 0 END) AS name,
       SUM(CASE WHEN dept       IS NULL THEN 1 ELSE 0 END) AS dept,
       SUM(CASE WHEN salary     IS NULL THEN 1 ELSE 0 END) AS salary,
       SUM(CASE WHEN join_date  IS NULL THEN 1 ELSE 0 END) AS join_date,
       SUM(CASE WHEN manager_id IS NULL THEN 1 ELSE 0 END) AS manager_id
FROM emp;

-- shorter ways for one column:
SELECT count_if(salary IS NULL)  AS salary_nulls FROM emp;   -- Spark / Databricks
SELECT COUNT(*) - COUNT(salary)  AS salary_nulls FROM emp;   -- works everywhere
```

Output

```
+------+----+----+------+---------+----------+
|emp_id|name|dept|salary|join_date|manager_id|
+------+----+----+------+---------+----------+
|0     |0   |0   |1     |0        |3         |
+------+----+----+------+---------+----------+
```

This is the one place where **PySpark is shorter than SQL**: the list comprehension loops over `df.columns`, so the same line works for 6 columns or 600. The trick: `isNull()` gives true/false, `.cast("int")` makes it 1/0, and `sum` adds them up, exactly the `SUM(CASE WHEN ... IS NULL THEN 1 ELSE 0 END)` pattern.

📝 **`COUNT(*)` vs `COUNT(column)`:** `COUNT(*)` counts rows (8). `COUNT(salary)` counts only **non-NULL** salaries (7). PySpark is identical: `F.count("*")` vs `F.count("salary")`. All aggregate functions (`SUM`, `AVG`, `MIN`, `MAX`) **ignore NULLs**.

PySpark

```
# drop rows with nulls
df.dropna().show()                       # any column null
df.dropna(subset=["salary"]).show()      # only salary
```

SQL

```
-- drop rows with nulls
SELECT * FROM emp
WHERE emp_id IS NOT NULL AND name IS NOT NULL AND dept IS NOT NULL
  AND salary IS NOT NULL AND join_date IS NOT NULL AND manager_id IS NOT NULL;
-- (any column null -> row removed)

SELECT * FROM emp WHERE salary IS NOT NULL;      -- only salary
```

Output

```
dropna() on our data keeps only rows with NO null anywhere:
+------+-----+-------+------+----------+----------+
|emp_id|name |dept   |salary|join_date |manager_id|
+------+-----+-------+------+----------+----------+
|2     |Ravi |IT     |75000 |2020-06-15|1         |
|2     |Ravi |IT     |75000 |2020-06-15|1         |
|4     |John |HR     |52000 |2022-09-20|3         |
|6     |Kiran|Finance|80000 |2021-07-30|5         |
+------+-----+-------+------+----------+----------+
```

Surprised? `dropna()` with no arguments also removes Asha, Meena and Sara because their **manager_id is NULL**. That is why you should almost always pass `subset=[...]` with the columns you actually care about.

PySpark

```
# fill nulls
df.fillna({"salary": 0}).show()

# fill with the average
avg_sal = df.select(F.avg("salary")).first()[0]
df.fillna({"salary": avg_sal}).show()

# first non-null value
df.withColumn("sal", F.coalesce("salary", F.lit(0))).show()
```

SQL

```
-- fill nulls
SELECT emp_id, name, dept, COALESCE(salary, 0) AS salary, join_date, manager_id
FROM   emp;

-- fill with the average  (scalar sub-query = a one-value query inside)
SELECT emp_id, name, dept,
       COALESCE(salary, (SELECT AVG(salary) FROM emp)) AS salary,
       join_date, manager_id
FROM   emp;

-- first non-null value
SELECT *, COALESCE(salary, 0) AS sal FROM emp;
```

Output

```
Tom's salary after filling with the average:
SQL    COALESCE(...)  ->  67428.57142857143
PySpark fillna(...)   ->  67428   (the float is cut to fit the integer column!)
```

⚠️ **Gotcha I found while testing:** `df.fillna({"salary": 67428.57})` on an **integer** column stores `67428` (the decimals are dropped), while SQL's `COALESCE` keeps `67428.571...`. If you want decimals, first `cast` the column to `double`, or use `F.coalesce`. Also, the average here is `67428.57` because the duplicate Ravi row is counted twice. After de-duplication it is `67500`.

⚠️ **Null is not equal to anything**, not even another null. `F.col("x") == None` does **not** work, and in SQL `x = NULL` is never true. Use `.isNull()` in PySpark and `IS NULL` in SQL.

PySpark

```
# null-safe equality: treats NULL == NULL as True
df.select(F.lit(None) == F.lit(None),            # -> NULL
          F.lit(None).eqNullSafe(F.lit(None)))   # -> true
```

SQL

```
SELECT NULL = NULL   AS normal_equal,     -- NULL  (unknown)
       NULL <=> NULL AS null_safe_equal;  -- true
```

Duplicates

PySpark

```
df.distinct().count()                     # whole row duplicates removed
df.dropDuplicates(["emp_id"]).count()     # based on chosen columns
```

SQL

```
SELECT COUNT(*) FROM (SELECT DISTINCT * FROM emp);        -- whole row duplicates removed

-- based on chosen columns (count of distinct ids)
SELECT COUNT(DISTINCT emp_id) FROM emp;
```

Output

```
distinct().count()              -> 7   (8 rows minus the duplicate Ravi)
dropDuplicates(["emp_id"]).count() -> 7
```

> `dropDuplicates(["emp_id"])` keeps an **arbitrary** row per key. If you need "keep the latest", use a **window function** (Section 7). In SQL, `SELECT DISTINCT emp_id, name` removes duplicate *combinations of the columns you list*, but you cannot pick which row survives without `ROW_NUMBER()` either.

From now on the guide uses a cleaned copy. In PySpark `df = df.dropDuplicates()`; in SQL we create a view called **`emp_clean`**:

PySpark

```
df_clean = df.dropDuplicates()
df_clean.createOrReplaceTempView("emp_clean")
```

SQL

```
CREATE OR REPLACE TEMP VIEW emp_clean AS
SELECT DISTINCT * FROM emp;
```

5️⃣ Summaries: `groupBy` + `agg`

The "GROUP BY" of SQL.

PySpark

```
df.groupBy("dept").agg(
    F.count("*").alias("headcount"),
    F.round(F.avg("salary"), 0).alias("avg_salary"),
    F.max("salary").alias("max_salary"),
    F.sum("salary").alias("total_salary"),
    F.countDistinct("emp_id").alias("unique_emps")
).orderBy(F.col("avg_salary").desc()).show()
```

SQL

```
SELECT dept,
       COUNT(*)                AS headcount,
       ROUND(AVG(salary), 0)   AS avg_salary,
       MAX(salary)             AS max_salary,
       SUM(salary)             AS total_salary,
       COUNT(DISTINCT emp_id)  AS unique_emps
FROM   emp
GROUP  BY dept
ORDER  BY avg_salary DESC;
```

Output

```
+-------+---------+----------+----------+------------+-----------+
|dept   |headcount|avg_salary|max_salary|total_salary|unique_emps|
+-------+---------+----------+----------+------------+-----------+
|Finance|2        |80000.0   |80000     |160000      |2          |
|IT     |4        |70000.0   |75000     |210000      |3          |
|HR     |2        |51000.0   |52000     |102000      |2          |
+-------+---------+----------+----------+------------+-----------+
```

Look at **IT**: `headcount = 4` (the duplicate Ravi counts) but `unique_emps = 3`. That is the difference between `COUNT(*)` and `COUNT(DISTINCT ...)`, in both languages. Also `AVG` ignores Tom's NULL: (60000 + 75000 + 75000) / 3 = 70000.

SQL aggregate	PySpark aggregate
`COUNT(*)`	`F.count("*")`
`COUNT(col)` (non-null)	`F.count("col")`
`COUNT(DISTINCT col)`	`F.countDistinct("col")`
`SUM`, `AVG`, `MIN`, `MAX`	`F.sum`, `F.avg`, `F.min`, `F.max`
`ROUND(x, 0)`	`F.round(x, 0)`
`collect_list(col)`	`F.collect_list("col")`

Aggregating without groups

PySpark

```
df.agg(F.avg("salary"), F.max("salary")).show()
```

SQL

```
SELECT AVG(salary), MAX(salary) FROM emp;       -- no GROUP BY = one row for the whole table
```

Collect values into a list

PySpark

```
df.groupBy("dept").agg(F.collect_list("name").alias("people")).show(truncate=False)
```

SQL

```
SELECT dept, collect_list(name) AS people
FROM   emp
GROUP  BY dept;

-- other databases:  STRING_AGG(name, ', ')  (PostgreSQL / SQL Server)
--                   GROUP_CONCAT(name)      (MySQL)
```

Output

```
+-------+----------------------+
|dept   |people                |
+-------+----------------------+
|IT     |[Asha, Ravi, Tom, Ravi]|
|HR     |[Meena, John]         |
|Finance|[Sara, Kiran]         |
+-------+----------------------+   (order inside the list is not guaranteed)
```

Filtering after aggregation (SQL's HAVING)

PySpark

```
(df.groupBy("dept")
   .agg(F.avg("salary").alias("avg_sal"))
   .filter(F.col("avg_sal") > 60000)
   .show())
```

SQL

```
SELECT dept, AVG(salary) AS avg_sal
FROM   emp
GROUP  BY dept
HAVING AVG(salary) > 60000;
```

Output

```
+-------+-------+
|dept   |avg_sal|
+-------+-------+
|IT     |70000.0|
|Finance|80000.0|
+-------+-------+
```

`WHERE` vs `HAVING` in PySpark: there is only one method, `.filter()`. Put it **before** `groupBy` and it behaves like `WHERE` (filters rows). Put it **after** `agg` and it behaves like `HAVING` (filters groups). Position decides.

> Remember: `groupBy` causes a **shuffle**.

Try it: Find the department with the highest average salary.

PySpark

```
(df_clean.groupBy("dept")
   .agg(F.round(F.avg("salary"), 0).alias("avg_sal"))
   .orderBy(F.col("avg_sal").desc())
   .limit(1)
   .show())
```

SQL

```
SELECT dept, ROUND(AVG(salary), 0) AS avg_sal
FROM   emp_clean
GROUP  BY dept
ORDER  BY avg_sal DESC
LIMIT  1;
```

Output

```
+-------+-------+
|dept   |avg_sal|
+-------+-------+
|Finance|80000.0|
+-------+-------+
```

6️⃣ Joins

Create a second table

We already created `depts` in Section 2. Quick reminder of both versions:

PySpark

```
depts = spark.createDataFrame(
    [("IT", "Bangalore"), ("HR", "Hyderabad"), ("Finance", "Mumbai"), ("Legal", "Delhi")],
    ["dept", "city"]
)
```

SQL

```
-- depts(dept, city) was created in Section 2
SELECT * FROM depts;
```

The join types you need

PySpark

```
df.join(depts, on="dept", how="inner").show()      # only matching
df.join(depts, on="dept", how="left").show()       # all from left
df.join(depts, on="dept", how="right").show()      # all from right
df.join(depts, on="dept", how="full").show()       # everything
df.join(depts, on="dept", how="left_anti").show()  # left rows with NO match
df.join(depts, on="dept", how="left_semi").show()  # left rows that HAVE a match (only left
columns)
```

SQL

```
SELECT * FROM emp e INNER JOIN      depts d USING (dept);                -- only matching
SELECT * FROM emp e LEFT  JOIN      depts d USING (dept);                -- all from left
SELECT * FROM emp e RIGHT JOIN      depts d USING (dept);                -- all from right
SELECT * FROM emp e FULL  JOIN      depts d USING (dept);                -- everything
SELECT * FROM emp e LEFT ANTI JOIN  depts d ON e.dept = d.dept;          -- left rows, NO
match
SELECT * FROM emp e LEFT SEMI JOIN  depts d ON e.dept = d.dept;          -- left rows, HAVE
match
```

> 🔑 **`on="dept"` in PySpark = `USING (dept)` in SQL.** Both keep **one** copy of the key column. If you write `ON e.dept = d.dept` with `SELECT *`, you get **two** `dept` columns. Pick the columns yourself: `SELECT e.*, d.city`.

Result of the right join / full join shows why they exist: Legal (no employees) only appears when the **right** table is kept:

PySpark

```
(df.join(depts, "dept", "full")
   .select("dept", "name", "city")
   .orderBy("dept", "name")
   .show())
```

SQL

```
SELECT COALESCE(e.dept, d.dept) AS dept, e.name, d.city
FROM   emp e FULL JOIN depts d ON e.dept = d.dept
ORDER  BY dept, name;
```

Output

```
+-------+-----+---------+
|dept   |name |city     |
+-------+-----+---------+
|Finance|Kiran|Mumbai   |
|Finance|Sara |Mumbai   |
|HR     |John |Hyderabad|
|HR     |Meena|Hyderabad|
|IT     |Asha |Bangalore|
|IT     |Ravi |Bangalore|
|IT     |Ravi |Bangalore|
|IT     |Tom  |Bangalore|
|Legal  |NULL |Delhi    |    <- department with no employees
+-------+-----+---------+
```

🍽️ Easy memory trick

Join	Think of it as	SQL without `ANTI`/`SEMI` keywords
inner	"Only guests who are on **both** lists"	`JOIN ... ON`
left	"All my guests, plus extra info if available"	`LEFT JOIN ... ON`
left_anti	"Guests who are on my list but **not** on yours"	`WHERE NOT EXISTS (SELECT 1 FROM b WHERE b.k = a.k)`
left_semi	"My guests who also appear on your list (filter only)"	`WHERE EXISTS (SELECT 1 FROM b WHERE b.k = a.k)`

`LEFT ANTI JOIN` and `LEFT SEMI JOIN` exist in Spark SQL, but **many other databases do not have them**. There you write `NOT EXISTS` / `EXISTS` (or `LEFT JOIN ... WHERE b.k IS NULL`). Avoid `NOT IN (subquery)` when the subquery column can contain NULLs: it returns **no rows** at all.

PySpark

```
depts.join(df, "dept", "left_anti").show()
```

SQL

```
SELECT d.*
FROM   depts d
WHERE  NOT EXISTS (SELECT 1 FROM emp e WHERE e.dept = d.dept);
```

Output

```
+-----+-----+
|dept |city |
+-----+-----+
|Legal|Delhi|
+-----+-----+
```

Join on different column names

PySpark

```
e = df.alias("e")
d = depts.withColumnRenamed("dept", "dept_name").alias("d")
e.join(d, F.col("e.dept") == F.col("d.dept_name"), "left").show()
```

SQL

```
SELECT e.*, d.city
FROM   emp e
LEFT   JOIN (SELECT dept AS dept_name, city FROM depts) d
       ON e.dept = d.dept_name;
```

`df.alias("e")` in PySpark = the table alias `emp e` in SQL. It lets you write `F.col("e.dept")` just like `e.dept`.

Self join (employee → manager)

PySpark

```
e = df.alias("e")
m = df.alias("m")

(e.join(m, F.col("e.manager_id") == F.col("m.emp_id"), "left")
  .select(F.col("e.name").alias("employee"), F.col("m.name").alias("manager"))
  .show())
```

SQL

```
SELECT e.name AS employee,
       m.name AS manager
FROM   emp e
LEFT   JOIN emp m
       ON e.manager_id = m.emp_id;
```

Output

```
+--------+-------+
|employee|manager|
+--------+-------+
|Ravi    |Asha   |
|Asha    |NULL   |
|Meena   |NULL   |
|John    |Meena  |
|Kiran   |Sara   |
|Tom     |Asha   |
|Ravi    |Asha   |     <- duplicate row, because we used emp (not emp_clean)
|Sara    |NULL   |
+--------+-------+   (row order is not guaranteed)
```

The same table is used **twice** with two different aliases (`e` = the employee, `m` = that employee's manager). `LEFT` keeps employees who have no manager (Asha, Meena, Sara).

⚠️ Duplicate column trap

If both tables have the same non-key column name, you get *ambiguity* errors. Fix by using `alias`, renaming, or joining with `on="col_name"` (string), which keeps one copy of the key. In SQL: use `USING (col)` or qualify every column (`e.dept`, `d.dept`).

🚀 Broadcast join (the biggest join optimization)

If one table is **small** (fits in memory), send a copy to every executor. No shuffle of the big table.

PySpark

```
from pyspark.sql.functions import broadcast
big.join(broadcast(small), "dept")
```

SQL

```
-- a "hint" inside a comment-style marker tells Spark what to do
SELECT /*+ BROADCAST(d) */ e.name, d.city
FROM   emp_clean e
JOIN   depts d ON e.dept = d.dept;
```

Output

```
Plan contains:  BroadcastHashJoin ... BuildRight
                +- BroadcastExchange ...  (the small table `depts` is copied to all
executors)
```

A SQL **hint** `/*+ BROADCAST(d) */` is the twin of `broadcast(d)`. The word inside the brackets is the **alias** of the small table. Spark auto-broadcasts tables smaller than `spark.sql.autoBroadcastJoinThreshold` (default 10 MB).

Try it: Find employees whose department does **not** exist in `depts`. (Hint: `left_anti`.)

PySpark

```
df.join(depts, "dept", "left_anti").show()
```

SQL

```
SELECT * FROM emp e
LEFT ANTI JOIN depts d ON e.dept = d.dept;
-- or:  SELECT * FROM emp e WHERE NOT EXISTS (SELECT 1 FROM depts d WHERE d.dept = e.dept)
```

Output

```
+------+----+----+------+---------+----------+
|emp_id|name|dept|salary|join_date|manager_id|
+------+----+----+------+---------+----------+
+------+----+----+------+---------+----------+   <- EMPTY
```

An empty result is a **correct answer** here: all employees belong to IT, HR or Finance, which are all in `depts`. (The opposite question, "departments with no employees", returns **Legal**: see Practice P5.)

7️⃣ Window Functions (Interview Gold 🏆)

A window function calculates **per group but keeps every row** (unlike `groupBy`, which collapses rows). In SQL this is the `OVER (...)` clause.

PySpark

```
from pyspark.sql.window import Window
```

SQL

```
-- nothing to import in SQL: the OVER (...) clause is part of the language
```

```
SQL      :  RANK() OVER ( PARTITION BY dept  ORDER BY salary DESC )
                          └──── the "window" ────┘
PySpark  :  F.rank().over( Window.partitionBy("dept").orderBy(F.col("salary").desc()) )
```

`PARTITION BY` = `partitionBy` (the groups). `ORDER BY` = `orderBy` (the order inside each group). `OVER` = `.over(w)`. In this section **we use the de-duplicated data** (`df_clean` / `emp_clean`).

1. Rank within a group

PySpark

```
w = Window.partitionBy("dept").orderBy(F.col("salary").desc())

df.withColumn("rank", F.rank().over(w)) \
  .withColumn("dense_rank", F.dense_rank().over(w)) \
  .withColumn("row_num", F.row_number().over(w)) \
  .show()
```

SQL

```
SELECT *,
       RANK()       OVER w AS rank,
       DENSE_RANK() OVER w AS dense_rank,
       ROW_NUMBER() OVER w AS row_num
FROM   emp_clean
WINDOW w AS (PARTITION BY dept ORDER BY salary DESC);  -- name it once
```

Output

```
(some columns hidden to fit the page)
+-----+-------+------+----+----------+-------+
|name |dept   |salary|rank|dense_rank|row_num|
+-----+-------+------+----+----------+-------+
|Kiran|Finance|80000 |1   |1         |1      |
|Sara |Finance|80000 |1   |1         |2      |  <- tie for 1st
|John |HR     |52000 |1   |1         |1      |
|Meena|HR     |50000 |2   |2         |2      |
|Ravi |IT     |75000 |1   |1         |1      |
|Asha |IT     |60000 |2   |2         |2      |
|Tom  |IT     |NULL  |3   |3         |3      |  <- NULL is last
+-----+-------+------+----+----------+-------+
```

The `WINDOW w AS (...)` clause in SQL is the twin of the variable `w = Window...` in PySpark: define once, use many times. When two rows tie (Kiran and Sara), `row_number` still has to pick an order, and it is **not guaranteed**; add a second sort column (like `emp_id`) to make it stable.

🥇 rank vs dense_rank vs row_number

Salaries: 80, 80, 70

Salary	row_number	rank	dense_rank
80	1	1	1
80	2	1	1
70	3	**3** (skips 2)	**2** (no gap)

2. Top N per group (very common!)

PySpark

```
w = Window.partitionBy("dept").orderBy(F.col("salary").desc())

(df_clean.filter(F.col("salary").isNotNull())
   .withColumn("rn", F.row_number().over(w))
   .filter(F.col("rn") <= 2)
   .drop("rn")
   .show())
```

SQL

```
SELECT emp_id, name, dept, salary, join_date, manager_id
FROM (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) AS rn
  FROM   emp_clean
  WHERE  salary IS NOT NULL
)
WHERE rn <= 2;
```

Output

```
+------+-----+-------+------+----------+----------+
|emp_id|name |dept   |salary|join_date |manager_id|
+------+-----+-------+------+----------+----------+
|6     |Kiran|Finance|80000 |2021-07-30|5         |
|5     |Sara |Finance|80000 |2018-11-05|NULL      |
|4     |John |HR     |52000 |2022-09-20|3         |
|3     |Meena|HR     |50000 |2019-01-10|NULL      |
|2     |Ravi |IT     |75000 |2020-06-15|1         |
|1     |Asha |IT     |60000 |2021-03-01|NULL      |
+------+-----+-------+------+----------+----------+
```

Why the subquery in SQL? A window function cannot be used directly in `WHERE` (the window is computed *after* `WHERE`). So you compute `rn` in an inner query, then filter in the outer query. PySpark does the same in two steps: `withColumn("rn", ...)` then `.filter(...)`. Databricks SQL (and some other engines) also offer a `QUALIFY` clause that does the filter in one query: `... QUALIFY ROW_NUMBER() OVER (...) <= 2`.

3. Keep the latest record per key (dedupe the smart way)

PySpark

```
w = Window.partitionBy("emp_id").orderBy(F.col("join_date").desc())
latest = df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
```

SQL

```
CREATE OR REPLACE TEMP VIEW latest AS
SELECT emp_id, name, dept, salary, join_date, manager_id
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY emp_id ORDER BY join_date DESC) AS rn
  FROM   emp
)
WHERE rn = 1;
```

Output

```
latest -> 7 rows (one per emp_id; the duplicate Ravi is gone, and
          if two rows had different dates, the newest one would win)
```

4. lag / lead (previous / next row)

PySpark

```
w = Window.partitionBy("dept").orderBy("join_date")

df_clean.withColumn("prev_salary", F.lag("salary", 1).over(w)) \
        .withColumn("next_salary", F.lead("salary", 1).over(w)) \
        .show()
```

SQL

```
SELECT name, dept, join_date, salary,
       LAG(salary, 1)  OVER w AS prev_salary,
       LEAD(salary, 1) OVER w AS next_salary
FROM   emp_clean
WINDOW w AS (PARTITION BY dept ORDER BY join_date);
```

Output

```
+-----+-------+----------+------+-----------+-----------+
|name |dept   |join_date |salary|prev_salary|next_salary|
+-----+-------+----------+------+-----------+-----------+
|Sara |Finance|2018-11-05|80000 |NULL       |80000      |
|Kiran|Finance|2021-07-30|80000 |80000      |NULL       |
|Meena|HR     |2019-01-10|50000 |NULL       |52000      |
|John |HR     |2022-09-20|52000 |50000      |NULL       |
|Ravi |IT     |2020-06-15|75000 |NULL       |60000      |
|Asha |IT     |2021-03-01|60000 |75000      |NULL       |
|Tom  |IT     |2023-01-12|NULL  |60000      |NULL       |
+-----+-------+----------+------+-----------+-----------+
```

`lag(col, 1)` looks **one row back**, `lead(col, 1)` looks **one row forward**, inside each group in the given order. The first row of each group has no previous row, so you get NULL.

5. Running total

PySpark

```
w = (Window.partitionBy("dept")
           .orderBy("join_date")
           .rowsBetween(Window.unboundedPreceding, Window.currentRow))

df_clean.withColumn("running_salary", F.sum("salary").over(w)).show()
```

SQL

```
SELECT name, dept, join_date, salary,
       SUM(salary) OVER (
         PARTITION BY dept
         ORDER BY join_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_salary
FROM   emp_clean;
```

Output

```
+-----+-------+----------+------+--------------+
|name |dept   |join_date |salary|running_salary|
+-----+-------+----------+------+--------------+
|Sara |Finance|2018-11-05|80000 |80000         |
|Kiran|Finance|2021-07-30|80000 |160000        |
|Meena|HR     |2019-01-10|50000 |50000         |
|John |HR     |2022-09-20|52000 |102000        |
|Ravi |IT     |2020-06-15|75000 |75000         |
|Asha |IT     |2021-03-01|60000 |135000        |
|Tom  |IT     |2023-01-12|NULL  |135000        |
+-----+-------+----------+------+--------------+
```

The frame `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` = `.rowsBetween(Window.unboundedPreceding, Window.currentRow)`: "from the first row of the group up to this row". Tom's NULL salary adds nothing, so the total stays at 135000.

> 📝 **Default frame:** if you give an `ORDER BY` but no frame, SQL and Spark use a *range* frame from the start to the current row, which treats rows with equal order values as one block. Writing `ROWS BETWEEN ...` explicitly avoids surprises. **No `ORDER BY` at all** = the frame is the whole partition.

6. Compare to the group average without collapsing

PySpark

```
w = Window.partitionBy("dept")

df_clean.withColumn("dept_avg", F.avg("salary").over(w)) \
        .withColumn("diff", F.col("salary") - F.col("dept_avg")) \
        .show()
```

SQL

```
SELECT name, dept, salary,
       AVG(salary) OVER (PARTITION BY dept)            AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept)   AS diff
FROM   emp_clean;
```

Output

```
+-----+-------+------+--------+-------+
|name |dept   |salary|dept_avg|diff   |
+-----+-------+------+--------+-------+
|Kiran|Finance|80000 |80000.0 |0.0    |
|Sara |Finance|80000 |80000.0 |0.0    |
|John |HR     |52000 |51000.0 |1000.0 |
|Meena|HR     |50000 |51000.0 |-1000.0|
|Asha |IT     |60000 |67500.0 |-7500.0|
|Ravi |IT     |75000 |67500.0 |7500.0 |
|Tom  |IT     |NULL  |67500.0 |NULL   |
+-----+-------+------+--------+-------+
```

This is the big win over `groupBy`: you get the **group average next to every row**, so you can compare each employee with their department without a join. The same result with `groupBy` needs a join back to the original table.

Try it: Find the 2nd highest salary in each department.

PySpark

```
w = Window.partitionBy("dept").orderBy(F.col("salary").desc())

(df_clean.filter(F.col("salary").isNotNull())
   .withColumn("dr", F.dense_rank().over(w))
   .filter("dr = 2")
   .show())
```

SQL

```
SELECT *
FROM (
  SELECT *,
         DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS dr
  FROM   emp_clean
  WHERE  salary IS NOT NULL
)
WHERE dr = 2;
```

Output

```
+------+-----+----+------+----------+----------+---+
|emp_id|name |dept|salary|join_date |manager_id|dr |
+------+-----+----+------+----------+----------+---+
|3     |Meena|HR  |50000 |2019-01-10|NULL      |2  |
|1     |Asha |IT  |60000 |2021-03-01|NULL      |2  |
+------+-----+----+------+----------+----------+---+
```

Finance has **no** 2nd highest salary: Kiran and Sara tie at 80000 and there is no lower salary, so no row gets `dr = 2`. That is the correct behaviour of `dense_rank`.

8️⃣ Spark SQL and UDFs

Use SQL when it feels natural

PySpark

```
df.createOrReplaceTempView("emp")

spark.sql("""
    SELECT dept, ROUND(AVG(salary), 0) AS avg_sal
    FROM emp
    GROUP BY dept
    HAVING AVG(salary) > 55000
    ORDER BY avg_sal DESC
""").show()
```

SQL

```
-- the same query, written directly in a SQL cell / %sql
SELECT dept, ROUND(AVG(salary), 0) AS avg_sal
FROM   emp
GROUP  BY dept
HAVING AVG(salary) > 55000
ORDER  BY avg_sal DESC;
```

Output

```
+-------+-------+
|dept   |avg_sal|
+-------+-------+
|Finance|80000.0|
|IT     |70000.0|
+-------+-------+
```

> DataFrame API and SQL compile to the **same plan**. Use whichever is clearer. A very common real-world style: write the heavy joins and aggregations in SQL, then continue with PySpark (`spark.sql(...).withColumn(...)`) for Python logic.

Three bridges to remember:

Direction	Code
DataFrame → SQL	`df.createOrReplaceTempView("emp")`
SQL → DataFrame	`result = spark.sql("SELECT ... FROM emp")`
Table name → DataFrame	`spark.table("emp")`

UDF (User Defined Function): use as a last resort

PySpark

```
from pyspark.sql.functions import udf

@udf(returnType=StringType())
def grade(s):
    if s is None: return "NA"
    return "A" if s >= 70000 else "B"

df.withColumn("grade", grade("salary")).show()
```

SQL

```
-- Register the Python UDF by name first (in Python):
--   spark.udf.register("grade", grade_fn, StringType())
-- then call it in SQL:
SELECT name, salary, grade(salary) AS grade
FROM   emp_clean;
```

Output

```
+-----+------+-----+
|name |salary|grade|
+-----+------+-----+
|Ravi |75000 |A    |
|Kiran|80000 |A    |
|Sara |80000 |A    |
|Asha |60000 |B    |
|Tom  |NULL  |NA   |
+-----+------+-----+   (some rows shown)
```

Why UDFs are slow ⚠️

- Spark can't optimize them (black box).
- Data moves between the JVM and Python for every row.

✅ Better: do the same with built-ins

PySpark

```
df.withColumn(
    "grade",
    F.when(F.col("salary").isNull(), "NA")
     .when(F.col("salary") >= 70000, "A")
     .otherwise("B")
)
```

SQL

```
SELECT name, salary,
       CASE WHEN salary IS NULL   THEN 'NA'
            WHEN salary >= 70000  THEN 'A'
            ELSE                       'B'
       END AS grade
FROM emp_clean;
```

> **Golden rule:** *Built-in function > pandas UDF > regular Python UDF.* In SQL terms: `CASE WHEN` is always better than a custom function, because Spark can see inside it and optimize it.

9️⃣ Performance Basics (what interviewers love)

Narrow vs wide transformations

Narrow (fast, no shuffle)	Wide (slow, shuffle)
`select`, `filter`, `withColumn`, `map`	`groupBy`, `join`, `distinct`, `orderBy`, `repartition`
SQL: `SELECT`, `WHERE`, `CASE`, scalar functions	SQL: `GROUP BY`, `JOIN`, `DISTINCT`, `ORDER BY`, window with `PARTITION BY`

Partitions

PySpark

```
df.rdd.getNumPartitions()      # how many partitions now?

df.repartition(8)              # increase/decrease WITH shuffle (even distribution)
df.repartition(8, "dept")      # by column
df.coalesce(2)                 # only DEcrease, NO full shuffle (cheaper)
```

SQL

```
-- SQL has no method call, so it uses "hints" after SELECT:
SELECT /*+ REPARTITION(8) */       * FROM emp;     -- like repartition(8)
SELECT /*+ REPARTITION(8, dept) */ * FROM emp;     -- like repartition(8, "dept")
SELECT /*+ COALESCE(2) */          * FROM emp;     -- like coalesce(2)
```

Output

```
I checked the partition counts on real Spark:
spark.sql("SELECT /*+ REPARTITION(8) */ * FROM emp").rdd.getNumPartitions()  -> 8
spark.sql("SELECT /*+ COALESCE(2) */    * FROM emp").rdd.getNumPartitions()  -> 2
```

	`repartition`	**`coalesce`**
Can increase partitions	✅	❌
Full shuffle	✅	❌ (mostly merges)
Use when	Need even spread, or more partitions	Reduce files before writing

> Thumb rule: aim for partitions of about **100-200 MB** each.

PySpark

```
spark.conf.set("spark.sql.shuffle.partitions", 8)
```

SQL

```
SET spark.sql.shuffle.partitions = 8;
```

`spark.sql.shuffle.partitions` (default **200**) controls partitions after shuffles. For small data, lower it (e.g., 8) to speed things up. Our `EXPLAIN` for `GROUP BY dept` after this setting shows `Exchange hashpartitioning(dept, 8)`: the shuffle really uses 8 partitions.

Cache / persist

If you reuse a DataFrame many times, store it in memory.

PySpark

```
df.cache()        # or df.persist()
df.count()        # first action fills the cache
# ... use df many times ...
df.unpersist()    # free memory when done
```

SQL

```
CACHE TABLE emp;            -- eager: loads into memory now
-- CACHE LAZY TABLE emp;    -- lazy: fills the cache on first use (like df.cache())
-- ... query emp many times ...
UNCACHE TABLE emp;          -- free memory when done
```

> Cache only what you reuse. Caching everything wastes memory.

Data skew 🐘

One key (e.g., `country = "India"`) has far more rows than others, so one task takes forever. Fixes: **salting** the key, **broadcast** the small side, or enable **AQE** (`spark.sql.adaptive.enabled=true`, on by default in Spark 3.2+).

Salting in plain words: split the one giant key into small pieces by adding a random number, join on (key + number), and the work spreads over many tasks. The small table is repeated once per number.

PySpark

```
SALTS = 4
big_salted   = big.withColumn("salt", (F.rand() * SALTS).cast("int"))
small_salted = small.withColumn("salt", F.explode(F.array([F.lit(i) for i in range(SALTS)])))

big_salted.join(small_salted, ["key", "salt"])
```

```sql
-- big side: a random salt 0..3 on every row
-- small side: repeat each row 4 times, once per salt value
SELECT b.*, s.attr
FROM  (SELECT *, CAST(rand() * 4 AS INT) AS salt FROM big) b
JOIN  (SELECT *, explode(sequence(0, 3)) AS salt FROM small) s
  ON  b.key = s.key AND b.salt = s.salt;
```

Other quick wins

1. Filter **early**, select only needed columns. *(SQL: put conditions in `WHERE`, avoid `SELECT *` on wide tables.)*
2. Prefer **Parquet** over CSV.
3. Prefer built-in functions over UDFs.
4. Avoid `collect()` on big data (it pulls everything to the driver and can crash it).
5. Use `explain()` to see the plan. *(SQL: `EXPLAIN SELECT ...`, or `EXPLAIN FORMATTED SELECT ...` for a readable version.)*

🎯 Top 25 Interview Questions (with crisp answers)

Each answer has the PySpark idea first and, where it helps, a **SQL angle** so you can explain it in both languages.

Basics

Q1. What is Spark and why is it faster than Hadoop MapReduce? Spark is a distributed data processing engine. It is faster because it keeps intermediate data **in memory** instead of writing to disk after every step, and it optimizes the whole plan (DAG).

Q2. What is PySpark? The Python API for Apache Spark. Your Python code builds a plan; the heavy work runs on the JVM executors. *SQL angle:* Spark SQL is another API to the same engine, so a SQL query and the equivalent DataFrame code produce the same plan.

Q3. RDD vs DataFrame vs Dataset?

- **RDD**: low-level, no schema, no optimizer.
- **DataFrame**: table-like with schema, optimized by Catalyst. **Use this.** (It is what a SQL table/view is in Spark.)
- **Dataset**: typed DataFrame, only in Scala/Java (not in Python).

Q4. What are transformations and actions? Transformations build a plan lazily (`filter`, `select`); actions trigger execution (`count`, `show`, `write`). *SQL angle:* `spark.sql("SELECT ...")` is lazy like a transformation; `INSERT`/`CREATE TABLE AS` and `.show()` are actions.

Q5. What is lazy evaluation and why is it useful? Spark waits until an action to run, so it can see the whole plan and optimize it (e.g., push filters down, skip unused columns).

Q6. What is a DAG? Directed Acyclic Graph: the plan of steps (stages) Spark builds from your transformations. You can print it with `df.explain()` or `EXPLAIN SELECT ...`.

Q7. What is a partition? A chunk of data processed by one task. More partitions = more parallelism (up to the number of cores).

Q8. Driver vs Executor? Driver: runs your main program, builds the plan, coordinates. Executors: run tasks on data partitions and store cached data.

Transformations

Q9. Narrow vs wide transformation? Narrow: each output partition depends on one input partition (no shuffle). Wide: data must move between partitions (shuffle). *SQL angle:* `WHERE`/`SELECT` are narrow; `GROUP BY`, `JOIN`, `ORDER BY`, `DISTINCT` are wide.

Q10. What is a shuffle and why is it expensive? Redistributing data across the network between executors. Costs disk I/O, network I/O, and serialization.

Q11. `repartition` vs `coalesce`? `repartition` can increase or decrease with a full shuffle. `coalesce` only decreases and avoids a full shuffle. *SQL angle:* hints `/*+ REPARTITION(n) */` and `/*+ COALESCE(n) */`.

Q12. `cache` vs `persist`? `cache()` = `persist()` with the default storage level. `persist` lets you choose levels (memory, disk, both). *SQL angle:* `CACHE TABLE t` / `UNCACHE TABLE t`.

Q13. `groupBy` vs window function? `groupBy` collapses rows to one per group. A window function keeps all rows and adds a computed value. *SQL angle:* `GROUP BY` vs `OVER (PARTITION BY ...)`.

Q14. `rank` vs `dense_rank` vs `row_number`? `row_number`: unique sequence. `rank`: ties share a rank, then gaps. `dense_rank`: ties share a rank, no gaps. (Same three functions in SQL.)

Q15. How do you find the Nth highest salary per department? `dense_rank()` over `Window.partitionBy("dept").orderBy(desc("salary"))`, then filter `rank == N`. *SQL:* `DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC)` in a subquery, then `WHERE dr = N`.

Q16. How do you remove duplicates but keep the latest? `row_number()` over `partitionBy(key).orderBy(desc(timestamp))`, then filter `rn == 1`. *SQL:* the same `ROW_NUMBER()` pattern inside a subquery, then `WHERE rn = 1`.

Joins and performance

Q17. What is a broadcast join? The small table is copied to all executors so the big table need not be shuffled. Use `broadcast(small_df)`. *SQL:* `SELECT /*+ BROADCAST(small) */ ...`.

Q18. How do you handle data skew? Broadcast the small side, salt the skewed key, or use Adaptive Query Execution (AQE).

Q19. Why avoid Python UDFs? They are opaque to the optimizer and need JVM↔Python serialization for each row. Use built-ins (`CASE WHEN` / `F.when`) or pandas UDFs.

Q20. Why is `collect()` dangerous? It brings all data to the driver's memory. On big data this causes out-of-memory errors. Use `take(n)`, `limit(n)` (SQL: `LIMIT n`), or write to storage.

Q21. Why Parquet over CSV? Columnar, compressed, schema included, supports column pruning and predicate pushdown. *SQL angle:* `SELECT col1 FROM parquet.`path`` reads only `col1`.

Q22. What is `spark.sql.shuffle.partitions`? Number of partitions created after a shuffle (default 200). Tune it based on data size. Set it with `spark.conf.set(...)` or `SET spark.sql.shuffle.partitions = 8;`.

Q23. What is AQE (Adaptive Query Execution)? Spark adjusts the plan at runtime: coalesces small shuffle partitions, switches join strategies, and handles skew. Enabled by default in Spark 3.2+.

Practical

Q24. How do you handle null values? Detect with `isNull()` (SQL: `IS NULL`), drop with `dropna()` (SQL: `WHERE ... IS NOT NULL`), fill with `fillna()` or `coalesce()` (SQL: `COALESCE`), and remember nulls never equal anything.

Q25. How would you debug a slow Spark job? Open the **Spark UI**: check stages with long tasks (skew), large shuffles, spills to disk; then use `explain()` (SQL: `EXPLAIN`), reduce shuffles, broadcast small tables, filter early, and tune partitions.

🏋️ Practice Set (with solutions)

Use the `df` (employees) and `depts` tables from above. Make sure you first clean `df` as: `df = df.dropDuplicates()`. In SQL, use the `emp_clean` view (made with `SELECT DISTINCT * FROM emp`). Row order may differ when you run it: Spark does not guarantee order without `ORDER BY`.

P1. Show employees earning more than the company average.

PySpark

```
avg_sal = df.select(F.avg("salary")).first()[0]
df.filter(F.col("salary") > avg_sal).show()
```

SQL

```
SELECT *
FROM   emp_clean
WHERE  salary > (SELECT AVG(salary) FROM emp_clean);
```

Output

```
+------+-----+-------+------+----------+----------+
|emp_id|name |dept   |salary|join_date |manager_id|
+------+-----+-------+------+----------+----------+
|2     |Ravi |IT     |75000 |2020-06-15|1         |
|6     |Kiran|Finance|80000 |2021-07-30|5         |
|5     |Sara |Finance|80000 |2018-11-05|NULL      |
+------+-----+-------+------+----------+----------+
```

In PySpark you first **fetch the average into a Python variable** (an action), then use it. In SQL the same idea is a **scalar sub-query** `(SELECT AVG(salary) ...)`. The company average is 66166.67, so Ravi (75000) and the two 80000 salaries pass.

P2. Department-wise headcount and average salary, highest first.

PySpark

```
(df.groupBy("dept")
   .agg(F.count("*").alias("headcount"), F.round(F.avg("salary"), 0).alias("avg_sal"))
   .orderBy(F.col("avg_sal").desc())
   .show())
```

SQL

```
SELECT dept,
       COUNT(*)              AS headcount,
       ROUND(AVG(salary), 0) AS avg_sal
FROM   emp_clean
GROUP  BY dept
ORDER  BY avg_sal DESC;
```

Output

```
+-------+---------+-------+
|dept   |headcount|avg_sal|
+-------+---------+-------+
|Finance|2        |80000.0|
|IT     |3        |67500.0|
|HR     |2        |51000.0|
+-------+---------+-------+
```

P3. Top 1 earner per department.

PySpark

```
w = Window.partitionBy("dept").orderBy(F.col("salary").desc_nulls_last())
df.withColumn("rn", F.row_number().over(w)).filter("rn = 1").show()
```

SQL

```
SELECT *
FROM (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC NULLS LAST) AS rn
  FROM   emp_clean
)
WHERE rn = 1;
```

Output

```
+------+-----+-------+------+----------+----------+---+
|emp_id|name |dept   |salary|join_date |manager_id|rn |
+------+-----+-------+------+----------+----------+---+
|6     |Kiran|Finance|80000 |2021-07-30|5         |1  |
|4     |John |HR     |52000 |2022-09-20|3         |1  |
|2     |Ravi |IT     |75000 |2020-06-15|1         |1  |
+------+-----+-------+------+----------+----------+---+
```

`desc_nulls_last()` = `DESC NULLS LAST`. Finance has a **tie** (Kiran and Sara, both 80000): `row_number` picks only one of them, and which one is not guaranteed. If you want **both** tied people, use `rank()` / `dense_rank()` and filter `= 1`.

P4. Employees with their manager's name.

PySpark

```
e, m = df.alias("e"), df.alias("m")
(e.join(m, F.col("e.manager_id") == F.col("m.emp_id"), "left")
  .select(F.col("e.name").alias("employee"), F.col("m.name").alias("manager"))
  .show())
```

SQL

```
SELECT e.name AS employee,
       m.name AS manager
FROM   emp_clean e
LEFT   JOIN emp_clean m ON e.manager_id = m.emp_id;
```

Output

```
+--------+-------+
|employee|manager|
+--------+-------+
|Asha    |NULL   |
|Ravi    |Asha   |
|Meena   |NULL   |
|John    |Meena  |
|Sara    |NULL   |
|Kiran   |Sara   |
|Tom     |Asha   |
+--------+-------+     (shown sorted by emp_id; your row order may differ)
```

P5. Departments that have no employees.

PySpark

```
depts.join(df, "dept", "left_anti").show()     # -> Legal
```

SQL

```
SELECT d.*
FROM   depts d
LEFT ANTI JOIN emp_clean e ON d.dept = e.dept;      -- -> Legal
-- or:  WHERE NOT EXISTS (SELECT 1 FROM emp_clean e WHERE e.dept = d.dept)
```

Output

```
+-----+-----+
|dept |city |
+-----+-----+
|Legal|Delhi|
+-----+-----+
```

P6. Replace null salaries with the department average.

PySpark

```
w = Window.partitionBy("dept")
df.withColumn("salary", F.coalesce("salary", F.avg("salary").over(w))).show()
```

SQL

```
SELECT emp_id, name, dept,
       COALESCE(salary, AVG(salary) OVER (PARTITION BY dept)) AS salary,
       join_date, manager_id
FROM   emp_clean;
```

Output

```
+------+-----+-------+-------+
|emp_id|name |dept   |salary |      (join_date, manager_id hidden)
+------+-----+-------+-------+
|1     |Asha |IT     |60000.0|
|2     |Ravi |IT     |75000.0|
|3     |Meena|HR     |50000.0|
|4     |John |HR     |52000.0|
|5     |Sara |Finance|80000.0|
|6     |Kiran|Finance|80000.0|
|7     |Tom  |IT     |67500.0|     <- (60000 + 75000) / 2 = 67500, the IT average
+------+-----+-------+-------+
```

A window function **inside** `COALESCE`: the average is computed per department and used only where `salary` is NULL. No join needed.

P7. Salary difference vs the previous joiner in the same department.

PySpark

```
w = Window.partitionBy("dept").orderBy("join_date")
df.withColumn("diff", F.col("salary") - F.lag("salary").over(w)).show()
```

SQL

```
SELECT *,
       salary - LAG(salary) OVER (PARTITION BY dept ORDER BY join_date) AS diff
FROM   emp_clean;
```

Output

```
+-----+-------+----------+------+------+
|name |dept   |join_date |salary|diff  |      (other columns hidden)
+-----+-------+----------+------+------+
|Sara |Finance|2018-11-05|80000 |NULL  |     <- first joiner: no previous row
|Kiran|Finance|2021-07-30|80000 |0     |
|Meena|HR     |2019-01-10|50000 |NULL  |
|John |HR     |2022-09-20|52000 |2000  |
|Ravi |IT     |2020-06-15|75000 |NULL  |
|Asha |IT     |2021-03-01|60000 |-15000|
|Tom  |IT     |2023-01-12|NULL  |NULL  |     <- NULL salary, so the difference is NULL
+-----+-------+----------+------+------+
```

P8. Word count (classic!).

PySpark

```
lines = spark.createDataFrame([("spark is fun",), ("spark is fast",)], ["line"])

(lines
 .select(F.explode(F.split("line", " ")).alias("word"))
 .groupBy("word").count()
 .orderBy(F.col("count").desc())
 .show())
```

SQL

```
SELECT word, COUNT(*) AS count
FROM (
  SELECT explode(split(line, ' ')) AS word
  FROM   VALUES ('spark is fun'), ('spark is fast') AS t(line)
)
GROUP  BY word
ORDER  BY count DESC;
```

Output

```
+-----+-----+
|word |count|
+-----+-----+
|is   |2    |
|spark|2    |
|fast |1    |
|fun  |1    |
+-----+-----+     (ties like "is" / "spark" can appear in either order)
```

`split(line, ' ')` makes an **array** of words; `explode` turns **each array element into its own row**; then it is a normal `GROUP BY word`. In classic SQL (no arrays) you would need a different technique, which is why this pattern is a favourite Spark interview question.

🧾 One-Page Cheat Sheet (PySpark ↔ SQL)

Task	PySpark	SQL
Start	`spark = SparkSession.builder.appName("x").getOrCreate()`	(session already exists in a SQL notebook)
Register a DataFrame	`df.createOrReplaceTempView("t")`	—
Read	`spark.read.csv(path, header=True, inferSchema=True)`	`CREATE TEMP VIEW v USING csv OPTIONS (path '..', header 'true')`
Read Parquet	`spark.read.parquet(path)`	`SELECT * FROM parquet.`path``
Write	`df.write.mode("overwrite").parquet(path)`	`INSERT OVERWRITE DIRECTORY 'path' USING parquet SELECT ...`
Inspect	`df.show(5); df.printSchema(); df.count(); df.columns`	`SELECT * FROM t LIMIT 5; DESCRIBE t; SELECT COUNT(*) FROM t`
Pick columns	`df.select("a", F.col("b").alias("c"))`	`SELECT a, b AS c FROM t`
Filter rows	`df.filter((F.col("a") > 1) & (F.col("b") == "x"))`	`WHERE a > 1 AND b = 'x'`
New column	`df.withColumn("new", F.col("a") * 2)`	`SELECT *, a * 2 AS new FROM t`
If / else	`F.when(cond, "Y").otherwise("N")`	`CASE WHEN cond THEN 'Y' ELSE 'N' END`
Drop / rename	`df.drop("a"); df.withColumnRenamed("a","b")`	list only wanted columns; `a AS b`
Sort	`df.orderBy(F.col("a").desc())`	`ORDER BY a DESC`
Clean	`df.dropna(subset=["a"]); df.fillna({"a":0}); df.dropDuplicates(["id"])`	`WHERE a IS NOT NULL; COALESCE(a, 0); SELECT DISTINCT ...`
Aggregate	`df.groupBy("k").agg(F.count("*"), F.sum("v"), F.avg("v"))`	`SELECT k, COUNT(*), SUM(v), AVG(v) FROM t GROUP BY k`
HAVING	`.agg(...).filter(F.col("x") > 1)`	`GROUP BY k HAVING AVG(v) > 1`
Join	`df1.join(df2, "key", "left")`	`FROM a LEFT JOIN b USING (key)`

Task	PySpark	SQL
Broadcast	`df1.join(broadcast(small), "key")`	`SELECT /*+ BROADCAST(s) */ ... FROM a JOIN small s ...`
Window	`w = Window.partitionBy("k").orderBy(F.col("v").desc())`	`OVER (PARTITION BY k ORDER BY v DESC)`
Rank / lag	`F.row_number().over(w); F.rank().over(w); F.lag("v").over(w)`	`ROW_NUMBER() OVER (...); RANK() OVER (...); LAG(v) OVER (...)`
Partitions	`df.repartition(8); df.coalesce(2)`	`/*+ REPARTITION(8) */`, `/*+ COALESCE(2) */`
Cache	`df.cache(); df.unpersist()`	`CACHE TABLE t; UNCACHE TABLE t`
Plan	`df.explain()`	`EXPLAIN SELECT ...`

⚠️ Top 10 Beginner Mistakes

1. Using `and` / `or` instead of `&` / `|` in filters. *(In SQL it is `AND` / `OR`.)*
2. Forgetting parentheses around each condition.
3. Comparing with `== None` instead of `.isNull()`. *(SQL: `= NULL` instead of `IS NULL`.)*
4. Calling `collect()` on large data.
5. Using Python UDFs when a built-in exists.
6. Using `inferSchema` in production.
7. Not importing `functions as F` (and clashing with Python's `sum` / `max`).
8. Caching everything.
9. Ignoring partitions and writing thousands of tiny files.
10. Expecting `show()` to be instant: it **is** an action, so it triggers the full work.

Extra mistakes SQL people make in PySpark

1. Writing `=` for equality in a filter. PySpark needs `==` (a single `=` is assignment in Python).
2. Writing the pipeline in SQL order (`select` first, then `filter`). In PySpark the order you write is the order it runs, so filter early.
3. Expecting a `SELECT` alias to be visible in `WHERE`. It is not (in classic SQL) and it is not in PySpark either.
4. Forgetting that `df.filter(...)` returns a **new** DataFrame. Nothing changes in `df` unless you assign the result (`df = df.filter(...)`).
5. Using `COUNT(salary)` and thinking it counts rows: it counts **non-NULL** values only (same in `F.count("salary")`).

🚀 Next Steps (after mastering this)

1. Build a mini project: **clean + analyze a Kaggle CSV** (e.g., Netflix, IPL, or NYC Taxi) and write results to Parquet. Do the same analysis once in SQL and once in PySpark and compare the answers.
2. Learn **Delta Lake** (ACID tables on top of Parquet).
3. Learn **Structured Streaming** basics.
4. Practice on **Databricks Community Edition** (free).
5. Read the **Spark UI** for every job you run.

> ⭐ **If this helped you, star the repo and share it with a friend.** *Consistency beats intensity. 30 minutes a day for 2 weeks = you are job-ready for PySpark basics.* 💪🔥

Happy Sparking! ⚡